

PPL LAB-8 EXERCISES

📅 Date @March 20, 2026

PROGRAMS ON MATRIX IN CUDA

NAME : DEVADATHAN N R

ROLL NO : 04

SECTION : CSE A1

REG NO : 230905010

Question 1:

Write a program in CUDA to add two Matrices for the following specifications

- Each row of the resultant matrix to be computed by one thread
- Each column of resultant matrix to be computed by one thread
- Each element of resultant matrix to be computed by one thread

Code:

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

__global__ void rowAddKernel(int *a,int *b,int *c,int m,int n){
    int row=threadIdx.x;
    if(row<m){
        for(int col=0;col<n;col++){
            c[row*n+col]=a[row*n+col]+b[row*n+col];
        }
    }
}

__global__ void colAddKernel(int *a,int *b,int *c,int m,int n){
    int col=threadIdx.x;
    if(col<n){
        for(int row=0;row<m;row++){
            c[row*n+col]=a[row*n+col]+b[row*n+col];
        }
    }
}

__global__ void elementAddKernel(int *a,int *b,int *c,int m,int n){
    int row=blockIdx.y*blockDim.y+threadIdx.y;
    int col=blockIdx.x*blockDim.x+threadIdx.x;

    if(row<m && col<n){
```

```

        c[row*n+col]=a[row*n+col]+b[row*n+col];
    }
}

void displayMat(int *m,int m_rows,int n_cols){
    for(int i=0;i<m_rows;i++){
        for(int j=0;j<n_cols;j++){
            printf("%d ",m[i*n_cols+j]);
        }
        printf("\n");
    }
}

int main(){
    int *h_a,*h_b,*h_c,m,n,*d_a,*d_b,*d_c;
    printf("Enter the number of rows (m): ");
    scanf("%d",&m);
    printf("Enter the number of columns (n): ");
    scanf("%d",&n);

    int size=m*n;

    h_a=(int*)malloc(size*sizeof(int));
    h_b=(int*)malloc(size*sizeof(int));
    h_c=(int*)malloc(size*sizeof(int));

    printf("Enter elements for matrix A:\n");
    for(int i=0;i<size;i++) scanf("%d",&h_a[i]);

    printf("Enter elements for matrix B:\n");
    for(int i=0;i<size;i++) scanf("%d",&h_b[i]);

    cudaMalloc((void**)&d_a,size*sizeof(int));
    cudaMalloc((void**)&d_b,size*sizeof(int));
    cudaMalloc((void**)&d_c,size*sizeof(int));

    cudaMemcpy(d_a,h_a,size*sizeof(int),cudaMemcpyHostToDevice);
    cudaMemcpy(d_b,h_b,size*sizeof(int),cudaMemcpyHostToDevice);

    // Row-wise addition
    rowAddKernel<<<1,m>>>>(d_a,d_b,d_c,m,n);
    cudaMemcpy(h_c,d_c,size*sizeof(int),cudaMemcpyDeviceToHost);
    printf("\nRow-wise result:\n");
    displayMat(h_c,m,n);

    // Column-wise addition
    colAddKernel<<<1,n>>>>(d_a,d_b,d_c,m,n);
    cudaMemcpy(h_c,d_c,size*sizeof(int),cudaMemcpyDeviceToHost);
    printf("\nColumn-wise result:\n");
    displayMat(h_c,m,n);
}

```

```

// Element-wise addition
dim3 threadsPerBlock(16,16);
dim3 blocksPerGrid(
    ceil((float)n/threadsPerBlock.x),
    ceil((float)m/threadsPerBlock.y)
);
elementAddKernel<<<blocksPerGrid,threadsPerBlock>>>(d_a,d_b,d_c,m,n);
cudaMemcpy(h_c,d_c,size*sizeof(int),cudaMemcpyDeviceToHost);
printf("\nElement-wise result:\n");
displayMat(h_c,m,n);

cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_c);

free(h_a);
free(h_b);
free(h_c);

return 0;
}

```

Output:

```

STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/08_Lab$ nvcc l8q1.cu -o l8q1
STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/08_Lab$ ./l8q1
Enter the number of rows (m): 3
Enter the number of columns (n): 3
Enter elements for matrix A:
1 2 3 4 5 6 7 8 9
Enter elements for matrix B:
1 2 3 4 5 6 7 8 9

Row-wise result:
2 4 6
8 10 12
14 16 18

Column-wise result:
2 4 6
8 10 12
14 16 18

Element-wise result:
2 4 6
8 10 12
14 16 18

```

Question 2:

Write a program in CUDA to multiply two matrices for the following specifications:

- Each row of resultant matrix to be computed by one thread
- Each column of resultant matrix to be computed by one thread
- Each element of resultant matrix to be computed by one thread

Code:

```

#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

__global__ void rowMulKernel(int *a,int *b,int *c,int wa,int wb){
    int row=threadIdx.x;
    for(int col=0;col<wb;col++){
        c[row*wb+col]=0;
        for(int k=0;k<wa;k++){
            c[row*wb+col]+=a[row*wa+k]*b[k*wb+col];
        }
    }
}

__global__ void colMulKernel(int *a,int *b,int *c,int ha,int wa){
    int col=threadIdx.x;
    for(int row=0;row<ha;row++){
        c[row*wa+col]=0;
        for(int k=0;k<wa;k++){
            c[row*wa+col]+=a[row*wa+k]*b[k*wa+col];
        }
    }
}

__global__ void elementMulKernel(int *a,int *b,int *c,int ha,int wb,int wa){
    int row=blockIdx.y*blockDim.y+threadIdx.y;
    int col=blockIdx.x*blockDim.x+threadIdx.x;

    if(row<ha && col<wb){
        c[row*wb+col]=0;
        for(int k=0;k<wa;k++){
            c[row*wb+col]+=a[row*wa+k]*b[k*wb+col];
        }
    }
}

void displayMat(int *m,int row,int col){
    for(int i=0;i<row;i++){
        for(int j=0;j<col;j++){
            printf("%d ",m[i*col+j]);
        }
        printf("\n");
    }
}

int main(){
    int *h_a,*h_b,*h_c,*d_a,*d_b,*d_c;
    int ha,wa,hb,wb;

```

```

printf("Enter dimensions of matrix A: ");
scanf("%d %d",&ha,&wa);
printf("Enter dimensions of matrix B: ");
scanf("%d %d",&hb,&wb);

if(wa!=hb){
    printf("Invalid dimensions for multiplication.\n");
    return -1;
}

int sizeA=ha*wa;
int sizeB=hb*wb;
int sizeC=ha*wb;

h_a=(int*)malloc(sizeA*sizeof(int));
h_b=(int*)malloc(sizeB*sizeof(int));
h_c=(int*)malloc(sizeC*sizeof(int));

printf("Enter elements for matrix A:\n");
for(int i=0;i<sizeA;i++) scanf("%d",&h_a[i]);

printf("Enter elements for matrix B:\n");
for(int i=0;i<sizeB;i++) scanf("%d",&h_b[i]);

cudaMalloc((void**)&d_a,sizeA*sizeof(int));
cudaMalloc((void**)&d_b,sizeB*sizeof(int));
cudaMalloc((void**)&d_c,sizeC*sizeof(int));

cudaMemcpy(d_a,h_a,sizeA*sizeof(int),cudaMemcpyHostToDevice);
cudaMemcpy(d_b,h_b,sizeB*sizeof(int),cudaMemcpyHostToDevice);

// Row-wise multiplication
rowMulKernel<<<1,ha>>>(d_a,d_b,d_c,wa,wb);
cudaMemcpy(h_c,d_c,sizeC*sizeof(int),cudaMemcpyDeviceToHost);
printf("\nRow-wise result:\n");
displayMat(h_c,ha,wb);

// Column-wise multiplication
colMulKernel<<<1,wb>>>(d_a,d_b,d_c,ha,wa);
cudaMemcpy(h_c,d_c,sizeC*sizeof(int),cudaMemcpyDeviceToHost);
printf("\nColumn-wise result:\n");
displayMat(h_c,ha,wb);

// Element-wise multiplication
dim3 threadsPerBlock(16,16);
dim3 blocksPerGrid(
    ceil((float)wb/threadsPerBlock.x),
    ceil((float)ha/threadsPerBlock.y)
);

```

```

    elementMulKernel<<<blocksPerGrid,threadsPerBlock>>>(d_a,d_b,d_c,ha,wb,wa);
    cudaMemcpy(h_c,d_c,sizeC*sizeof(int),cudaMemcpyDeviceToHost);
    printf("\nElement-wise result:\n");
    displayMat(h_c,ha,wb);

    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_c);
    free(h_a);
    free(h_b);
    free(h_c);

    return 0;
}

```

Output:

```

STUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/08_Lab$ nvcc l8q2.cu -o l8q2
^[[A^[[ASTUDENT@MIT-ICT-LAB5-4:~/Desktop/230905010/08_Lab$ ./l8q2
Enter dimensions of matrix A: 3 3
Enter dimensions of matrix B: 3 3
Enter elements for matrix A:
1 2 3 4 5 6 7 8 9
Enter elements for matrix B:
1 2 3 4 5 6 7 8 9

Row-wise result:
30 36 42
66 81 96
102 126 150

Column-wise result:
30 36 42
66 81 96
102 126 150

Element-wise result:
30 36 42
66 81 96
102 126 150

```